

Working Title Placeholder

Author Name¹

Institution Name, City, Country
author@example.com

Abstract. This paper presents the current DNAT marketplace architecture as a security-oriented design for confidential data processing. The implementation combines a blockchain-based access-control layer, IPFS-based artifact distribution, a dedicated build virtual machine, and a dedicated execution virtual machine backed by Firecracker microVMs. The main architectural argument is that the system reduces blast radius by separating marketplace logic, dependency resolution, and data processing into different trust zones with distinct persistence rules and network capabilities. In particular, the execution path avoids outbound network access and relies on short-lived microVM instances, while the build path restricts reuse to a wheel cache instead of persisting full applications. The paper argues that simpler alternatives, such as a single control-and-execution host or a shared build-and-run environment, would enlarge the attack surface and make lateral movement, persistence, and data exfiltration easier. At the same time, the current design introduces operational cost, latency, and residual risks that must be discussed explicitly, including the absence of mutual authentication between internal services, the trust concentration that remains in the control plane, and the need for a stronger confidentiality story for dataset handling. The goal of the paper is therefore not to claim a perfect trusted execution platform, but to justify why the present decomposition is a defensible security baseline for a confidential marketplace prototype.

Keywords: Confidential computing · Data marketplace · Firecracker · Isolation · Security architecture

1 Introduction

Confidential data marketplaces must support two goals that naturally conflict: they need to preserve the economic and governance properties of data sharing while reducing the risk that uploaded code, third-party dependencies, or execution results expose sensitive assets. In practice, the main difficulty is not only protecting data at rest, but also constraining what a buyer-provided application can reach during build and execution. The architecture implemented in this repository addresses this problem by decomposing the platform into a control plane, an isolated build plane, and an isolated execution plane.

The paper should defend the following thesis: the current three-layer design is justified because it reduces privilege concentration and constrains failure

propagation better than simpler alternatives. The central claim is not that each component is individually invulnerable, but that compromise in one stage does not automatically grant access to all other assets. This matters in data marketplaces because the system must simultaneously handle datasets, application artifacts, dependency resolution, access checks, execution results, and persistent metadata.

This draft starts from three contributions:

- an implementation-oriented description of the current DNAT architecture, with emphasis on trust boundaries and persistence rules;
- a security argument explaining why build isolation and execution isolation should remain separate concerns;
- a discussion of residual weaknesses and alternative architectures that would introduce additional attack paths.

2 System Overview

The current implementation is organized around three components. **CVM1** hosts the client-facing logic, including the local marketplace interface, the blockchain interaction layer, IPFS integration, and artifact encryption before application publication. **CVM2** is a dedicated build service that receives application bundles from CVM1 and spawns an ephemeral Firecracker microVM to resolve dependencies and produce an `application.ext4` artifact. **CVM3** is a dedicated execution service that receives a temporary execution bundle and launches an ephemeral Firecracker microVM without network access.

This separation is important because each component persists different classes of data. CVM1 retains marketplace state and published blobs, CVM2 persists only reusable wheel files, and CVM3 is reduced to runtime artifacts needed to launch isolated executions. Temporary data are removed after build and execution, limiting cross-request contamination. The repository documentation already reflects this design through architecture diagrams, build scripts, and runtime cleanup procedures.

From a workflow perspective, application registration and application execution are intentionally decoupled. Registration sends the source bundle to the build plane, stores the built application artifact, and publishes only the resulting encrypted blob. Execution later retrieves the dataset and application artifact through the control plane, prepares a temporary workspace, and forwards only the execution bundle to the isolated runtime. This distinction is essential to the security argument because build-time code and run-time data are not treated as the same trust problem.

3 Threat Model and Assumptions

The main adversarial scenarios considered in this work are the following. First, a malicious application author may try to use the submitted code or dependency graph to escape the execution environment, exfiltrate datasets, or persist

malicious state across runs. Second, a malicious or compromised dependency may attempt supply-chain abuse during the build phase. Third, an attacker who compromises a less privileged component may try to move laterally toward marketplace metadata, stored assets, wallet material, or future executions.

The design partially addresses these scenarios by constraining component reachability and by minimizing what each layer can store or access. However, the implementation does *not* currently remove all trust from the infrastructure. The control plane remains privileged, internal communication does not yet implement mutual TLS, and the repository does not expose a complete remote attestation flow that would let an external party verify the precise software state of all confidential components before use. Therefore, the paper should avoid absolute claims about end-to-end trustlessness.

Another important assumption concerns dataset handling. The current code clearly encrypts the built application artifact before it is published to IPFS, but dataset confidentiality depends on how the dataset is prepared before registration and on how the control plane handles it during execution orchestration. This should be stated as an implementation limitation or as a deployment assumption, not hidden behind generic confidentiality claims.

4 Architectural Rationale

4.1 Why the Control Plane Should Not Build Applications

Keeping dependency resolution outside CVM1 is a strong design choice. The control plane is the most security-sensitive part of the system because it touches marketplace state, IPFS operations, access validation, and the logic that orchestrates publication and execution. If dependency installation happened in the same component, any malicious package, build hook, or native extension would execute in the same administrative context as the marketplace layer. That would turn a supply-chain problem into a platform compromise.

The current design breaks that path. CVM2 exposes only a minimal build API, persists only wheel files, and uses an ephemeral worker plus an ephemeral build microVM. Even when outbound network access is needed to download packages, the build script explicitly blocks access to private address ranges on the host side. This does not eliminate supply-chain risk, but it narrows the assets that a successful build compromise can reach.

4.2 Why the Execution Plane Should Stay Separate from the Build Plane

Combining build and execution into the same isolated host would look simpler, but it would also couple two threat classes that should remain distinct. Build logic needs egress to download dependencies, whereas the execution path should be as closed as possible because it handles the dataset-processing step. If the same environment is reused for both phases, the runtime inherits more tools, more state, and more opportunities for persistence than necessary.

The current implementation avoids that coupling. The execution microVM does not receive a network interface, mounts the application artifact read-only, uses a temporary input bundle, and returns results through a persistent output disk collected by the host after shutdown. This sharply limits outbound exfiltration opportunities and keeps the runtime contract narrow: execute the packaged workload, write the result, and terminate.

4.3 Why Ephemeral State Matters

The architecture depends heavily on the distinction between persistent and ephemeral data. The build plane keeps wheel files but discards the application state after each request. The execution plane removes temporary overlays, disks, mounts, and the Firecracker socket after each run. These cleanup rules are not cosmetic details; they are central to the argument that compromise impact should be bounded in time and scope.

This ephemeral strategy also creates a cleaner narrative for the paper. Instead of claiming that confidential execution magically solves all security issues, the system can be presented as a layered containment strategy: reduce what is reachable during each phase, reduce what survives each phase, and reduce what each compromised component can reveal about the rest of the platform.

5 Security Discussion

5.1 Positive Aspects of the Current Design

The first major advantage is **blast-radius reduction**. A compromise in the execution microVM does not automatically expose the local IPFS node, the blockchain client, or the build cache because those resources are outside the execution environment. The second advantage is **phase-specific hardening**. The build phase tolerates limited outbound connectivity but blocks private destination ranges, while the execution phase removes network access entirely. The third advantage is **minimal persistence**. The build plane persists only wheel files, and the execution plane is designed to die cleanly after each run.

The fourth advantage is **artifact immutability during execution**. The execution host mounts `application.ext4` read-only, which reduces the risk that a single run silently mutates the packaged application for later reuse. The fifth advantage is **narrow interfaces**. Both the build plane and the execution plane are invoked through small service APIs instead of exposing general-purpose shells or shared storage to the control plane.

5.2 Negative Aspects and Residual Risk

The current architecture also has non-trivial drawbacks. The first is **operational complexity**. Three components, temporary disks, Firecracker orchestration, and containerized deployment increase engineering and debugging cost. The

second is **latency overhead**. Build and execution both pay setup and teardown costs that a monolithic design would avoid. The third is **control-plane trust concentration**. CVM1 still orchestrates the workflow and remains a high-value target.

There are also security gaps that should be discussed honestly. Internal service authentication is still incomplete because the repository documentation explicitly notes that mutual TLS is not implemented. Dataset confidentiality handling is weaker than the application-artifact path and should be tightened before making stronger confidentiality claims. Finally, wheel caching improves performance, but it remains a supply-chain sensitive mechanism and should be treated as a controlled optimization rather than a free security win.

5.3 Why Simpler Alternatives Increase the Attack Surface

A single-VM architecture would collapse trust boundaries and make lateral movement easier. Malicious code could target the same filesystem namespace that contains marketplace data, execution outputs, IPFS state, or private blockchain material. A shared build-and-run environment would also let build-time tools survive long enough to affect later executions. Likewise, enabling network access inside the execution microVM would reintroduce exfiltration channels, command-and-control paths, and metadata probing opportunities that the current executor intentionally avoids.

Figure 4 is useful precisely because it makes the alternative design look attractive at first: fewer machines, fewer APIs, and less operational friction. However, that simplicity comes from collapsing roles that should remain separate. In the centralized layout, the execution orchestrator, locally stored artifacts, IPFS-facing logic, and sensitive wallet-related material live close enough that a successful compromise can pivot across responsibilities with much less resistance. By contrast, Fig. 3 shows that the current design treats build, execution, and control as different failure domains.

The most important argumentative move for the paper is therefore comparative rather than absolute: the current architecture should be defended because it is safer than simpler alternatives under realistic implementation constraints, not because it eliminates every attack class.

6 Attack Scenarios and Current Mitigations

7 Evaluation Plan

The paper can evolve from a purely architectural description into a practical security study by validating a small set of claims experimentally. First, the build path can be tested with a deliberately hostile dependency or build script to show that compromise is contained inside the build plane and does not grant direct access to marketplace assets. Second, the runtime can be tested with an application that attempts outbound network communication, proving that the

Table 1. Starter matrix for the paper’s security discussion.

Attack	Why it matters	Current architectural answer
Dependency-based compromise during build	Malicious packages may execute arbitrary code while dependencies are resolved.	Build happens in CVM2 through an ephemeral worker and an ephemeral build microVM; private host ranges are blocked during package download; only wheel files are retained after the build.
Runtime data exfiltration	Buyer code may try to leak sensitive dataset contents.	The execution microVM is launched without a network interface and returns results only through a collected output disk.
Cross-run persistence	One execution may try to plant tools or modified state for future runs.	Temporary overlays, input disks, output disks, mounts, and Firecracker sockets are removed after execution; the application artifact is mounted read-only.
Lateral movement toward market-place services	A runtime compromise may attempt to reach IPFS, wallet material, or smart-contract logic.	Execution is separated from the control plane, and the executor does not host marketplace services or IPFS.
Cache poisoning or unsafe artifact reuse	Build optimizations may become a persistence or tampering channel.	CVM2 ingests only <code>.whl</code> files, validates names, rejects oversized files, and prunes the cache by size; however, stronger provenance validation is still desirable.
Internal service impersonation	A forged internal service could feed malicious build or execution results.	This remains a gap in the current repository because mutual TLS and stronger service identity are still pending.

current executor blocks that behavior by construction. Third, persistence claims can be validated by checking that temporary execution artifacts disappear after each run.

It is also useful to evaluate the tradeoff side of the design. The paper should measure build latency, execution latency, and artifact size overhead introduced by the isolation layers. Those numbers will help justify the claim that the architecture buys meaningful risk reduction at an acceptable engineering cost for a confidential marketplace setting.

The experimental section can be strengthened with security testing tools that target different layers of the system. Static analyzers such as Semgrep and Bandit can inspect the orchestration code for unsafe patterns. Supply-chain tools such as pip-audit, Syft, and Gripe can expose dependency and artifact risk in the build path. Container and image scanners such as Trivy are relevant for the CVM images themselves. Dynamic tools such as Nmap, curl, and custom proof-of-concept workloads can validate that the executor remains unreachable from the outside and that the guest cannot perform the outbound actions that a less isolated architecture would allow.

8 Conclusion

This draft frames the DNAT implementation as a layered containment architecture rather than a generic marketplace prototype. Its main value is the explicit separation of control, build, and execution concerns, combined with ephemeral microVM-based processing and constrained persistence. That separation provides a defensible answer to attacks involving untrusted application code, dependency resolution, and lateral movement across platform responsibilities.

At the same time, the architecture should be presented with precise limits. The control plane is still privileged, internal mutual authentication is incomplete, and dataset confidentiality deserves stronger treatment in the implementation. These limitations do not weaken the paper if they are discussed directly; instead, they make the argument more credible by showing that the design is security-conscious without overstating what has already been achieved.

References

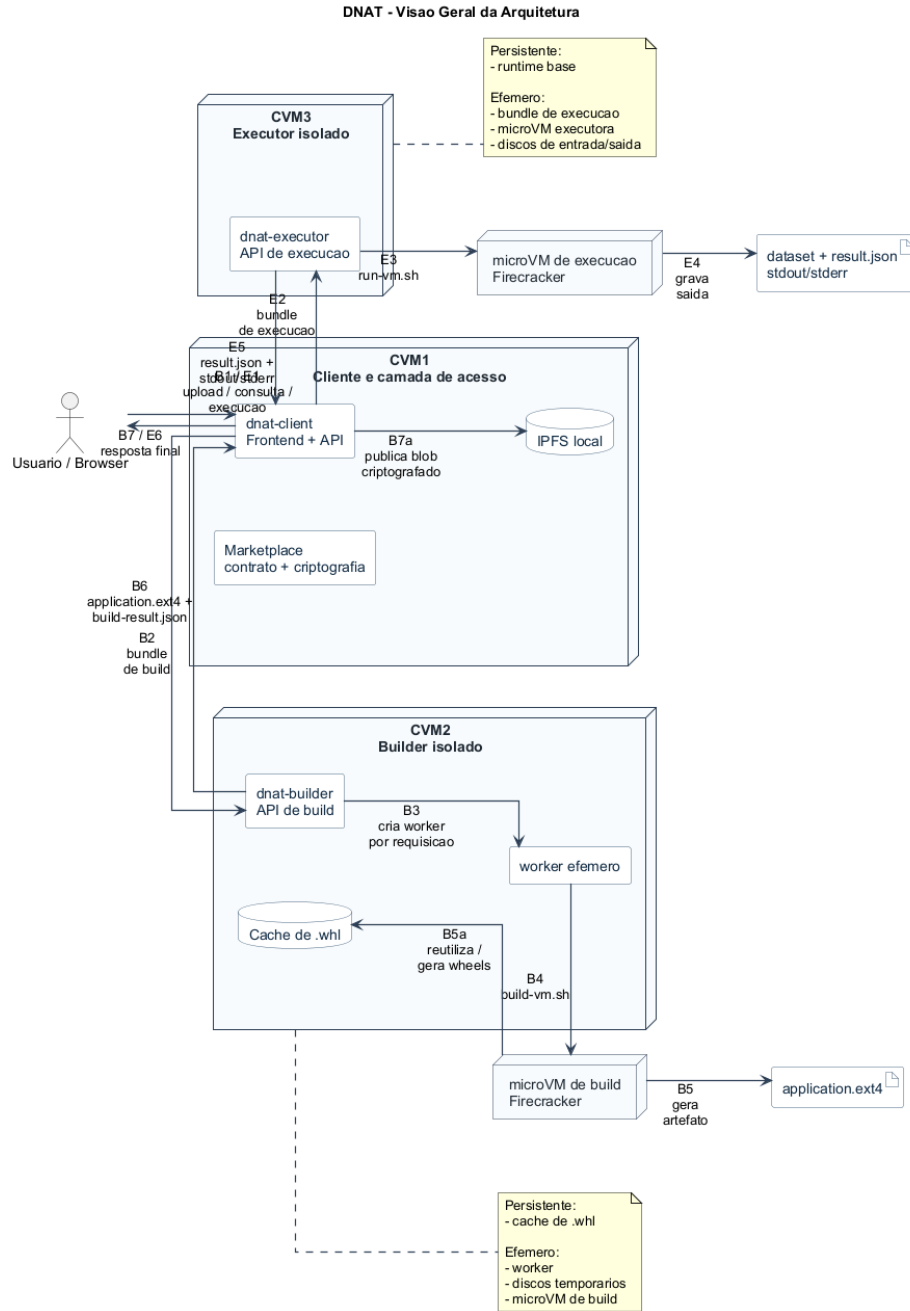


Fig. 1. High-level view of the current DNAT architecture. The main value of this figure is clarity: it shows the separation between the control plane, the isolated build plane, and the isolated execution plane.

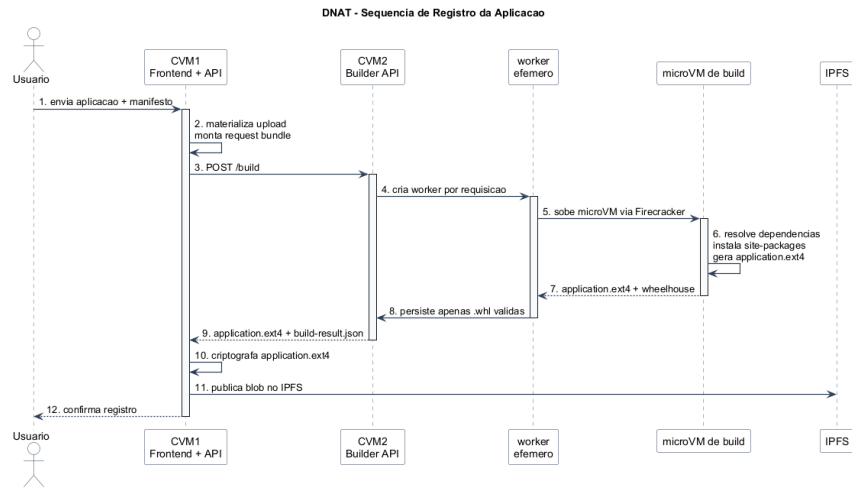


Fig. 2. Application registration flow. This figure is useful because it makes the build isolation argument concrete: dependency resolution occurs in the builder path before the encrypted artifact is published.

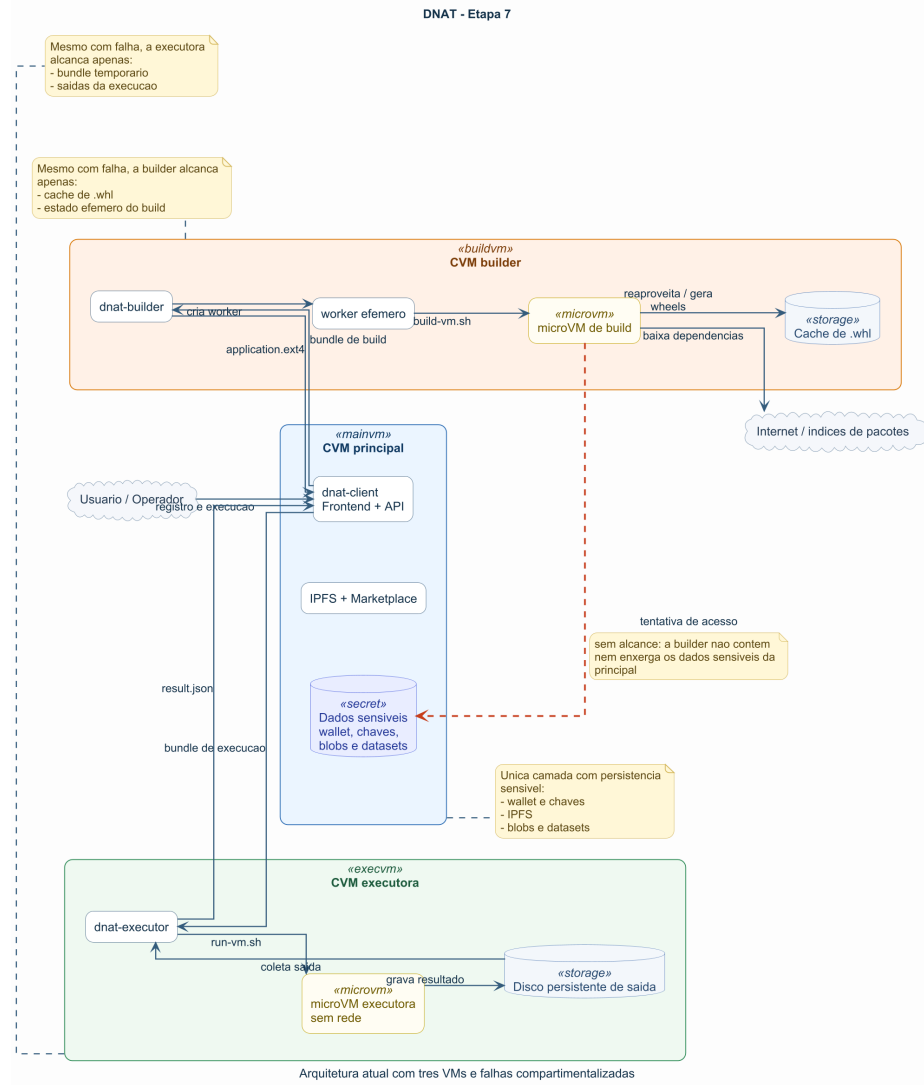


Fig. 3. Final compartmentalized architecture. This figure supports the core claim of the paper: compromise in the build or execution plane does not immediately expose the full set of sensitive assets held by the control plane.

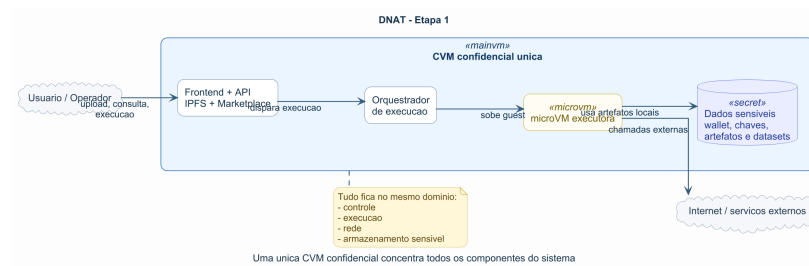


Fig. 4. Centralized baseline architecture. This is the comparison the paper needs: control logic, sensitive storage, and execution all remain in the same trust domain, so a local compromise has a much larger blast radius.